

EXERCISE I

In reference to the Stochastic Gradient Descent (SDG) algorithm

I.a) Explain the concept of loss function and provide one example

I.b) Does the gradient descent algorithm guarantee global minimization of the loss function? Motivate your answer

I.c) Assume you train a neural network for 1000 epochs. Backpropagation requires $(0.001 \cdot N_{\text{sample}} + 0.2)$ seconds per epoch. You have a training set with 15.000 samples

and a sampling ratio of 15% for the stochastic selection of the samples. Compute total training time.

epochs = 1000 # Total epochs

N_samples = 15000 # Total number of samples

sampling_ratio = 0.15 # Sampling ratio for stochastic selection

$\text{samples_per_epoch} = N_samples * \text{sampling_ratio} = 15000 * 0.15 = 2250$

$\text{time_per_epoch} = \text{samples_per_epoch} * 0.001 + 0.2 \text{ [s]} = 2.45 \text{ s}$

Computing the total training time

$\text{total_time} = \text{time_per_epoch} * \text{epochs} = 2.45\text{s} * 1000 = 2450 \text{ s}$

EXERCISE II

Consider a convolutional neural network layer with the following characteristics: 64 input channels (40x40), convolutional filters size 5x5, 256 output channels. No padding and stride 1.

II.a) Compute the total storage size needed for input, output activations and weights assuming activations are 16bit and weights are 8bit.

II.b) Compute the total number of MAC operations needed to compute all the output channels in the layer.

II.c) Compute the number of SUM and the number of DIV operations needed to perform average pooling (3x3) on all the output channels, consider pooling stride = 3.

EXERCISE III

A parallel cluster of RISC-V processors is executing a convolutional layer with 9 input channels and 18 output channels. The input spatial size is 64x64. Inputs, outputs, and weights are quantized to 16 bits. The convolutional filter size is 3x3, no padding is used. RELU non-linearity is applied in place on the output. Assume that on average a RISC-V processor can perform a MAC in 4 cycles and a RELU operation in 6 cycles. RELU and convolutions are parallelized in an ideal fashion, with no overhead and perfect load balancing.

III.a) Compute the L1 minimal shared memory size that is needed to store the complete layer (input and output channels as well as weights) in L1

III.b) Compute the total number of cycles to process the layer on a 4-core cluster

	cycles	Processed by	
MAC	4 cycles	4	Cores
ReLU	6 cycles	4	Core

$\text{Cycles_conv (4 cores)} = (\text{Hout} * \text{Wout} * \text{Cin} * (\text{Cout}/\text{Ncores}) * K * K) * [\text{MAC}] \text{ cycles} = 5604552 \text{ cycles}$
 $\text{Cycles_ReLU (4 core)} = ((\text{Cout}/\text{N_cores}) * \text{Wout} * \text{Hout}) * [\text{ReLU}] \text{ cycles} =$
 $= (19/\text{N_cores} * 126 * 126) * [\text{ReLU}] \text{ cycles} = 103788 \text{ cycles}$
Cycles_tot = 5708340 cycles

Note: if the parallelization was not ideal, then:

$\text{Cycles_conv (4 cores)} = (\text{Hout} * \text{Wout} * \text{Cin} * \text{ceil}(\text{Cout}/\text{Ncores}) * K * K) * [\text{MAC}] \text{ cycles} = 6227280 \text{ cycles}$
 $\text{Cycles_ReLU (4 core)} = (\text{ceil}(\text{Cout}/\text{N_cores}) * \text{Wout} * \text{Hout}) * [\text{ReLU}] \text{ cycles} =$
 $= (19/\text{N_cores} * 126 * 126) * [\text{ReLU}] \text{ cycles} = 115320 \text{ cycles}$
Cycles_tot = 6342600 cycles

III.c) Compute the speedup with respect to sequential when the cluster has 8 cores

Sequential:

	cycles	Processed by	
MAC	4 cycles	1	Cores
ReLU	6 cycles	1	Core

$\text{Cycles_conv (4 cores)} = (\text{Hout} * \text{Wout} * \text{Cin} * (\text{Cout}/\text{Ncores}) * K * K) * [\text{MAC}] \text{ cycles} = 22418208 \text{ cycles}$
 $\text{Cycles_ReLU (4 core)} = ((\text{Cout}/\text{N_cores}) * \text{Wout} * \text{Hout}) * 3 \text{ cycles} =$
 $= (19/\text{N_cores} * 126 * 126) * [\text{ReLU}] \text{ cycles} = 415152 \text{ cycles}$
Cycles_tot = 22833360 cycles

8 cores:

	cycles	Processed by	
MAC	4 cycles	8	Cores
ReLU	6 cycles	8	Core

Cycles_conv (4 cores) = (Hout * Wout * Cin * (Cout/Ncores) * K * K) * [MAC] cycles = 2802276 cycles

Cycles_ReLU (4 core) = ((Cout/N_cores) *Wout*Hout) * 3 cycles =
= (19/N_cores*126*126)* [ReLU] cycles = 51894 cycles

Cycles_tot = **2854170 cycles**

Speedup= 22833360 /2854170 = 8x